ServiceNow SAM — Request Fulfillment & License Reclamation Workflow

Implementation Solution Guide

Version 1.0.0 · 2026-06-29

Contents

Executive Summary — ServiceNow SAM Request Fulfillment & License Reclamation Workflow . . . 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 4

1. Solution Overview — ServiceNow SAM Request Fulfillment & License Reclamation Workflow . . . 5

What this solution is . . . 5

Why this matters (the why — covered only here) . . . 5

The request lifecycle . . . 5

Named artifacts (use these exact names everywhere) . . . 5

What you can control . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow . . . 7

2.1 ServiceNow instance with SAM / ITAM . . . 7

2.2 Administrative / SAM roles . . . 7

2.3 Entitlement data loaded . . . 7

2.4 Software model / catalog content . . . 8

2.5 Discovery / usage data integration . . . 8

2.6 Approval owners and groups . . . 8

2.7 Reporting / KPI surface . . . 8

2.8 Pilot scope defined . . . 8

Checklist . . . 8

3. Implementation Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow . . . 9

Step 1 — Create the groups . . . 9

Step 2 — Build the intake catalog items . . . 9

Step 3 — Build the triage flow . . . 10

Step 4 — Build the entitlement check flow . . . 10

Step 5 — Build the fulfillment flow . . . 10

Step 6 — Build the reclamation rules and flow . . . 10

Step 7 — Build the KPI set . . . 11

Step 8 — Apply and validate on the pilot . . . 11

Step 9 — Enable reclamation, then expand . . . 11
5. Testing Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow 12
Test 1 — Intake captures a tracked request . . . 12
Test 2 — Triage routes correctly . . . 12
Test 3 — Entitlement check is accurate . . . 12
Test 4 — Fulfillment records the assignment . . . 12
Test 5 — Reclamation flags the right candidates (report-only) . . . 12
Test 6 — Reclamation returns units (enforce mode) . . . 12
Test 7 — KPI set reports throughput and yield . . . 12
Pass/fail gate . . . 13
6. Troubleshooting Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow 13
A. Requests not flowing through triage . . . 13
B. Entitlement check wrong . . . 13
C. Fulfillment not recording . . . 14
D. Reclamation flags active users (false positives) . . . 14
E. KPI set empty or wrong . . . 14
F. Diagnostics quick reference . . . 14
G. When to escalate . . . 14
7. Best Practices — ServiceNow SAM Request Fulfillment & License Reclamation Workflow 15
8. Deliverables Summary — ServiceNow SAM Request Fulfillment & License Reclamation Workflow 16
Documentation . . . 16
PowerShell / integration scripts . . . 16
What the buyer receives . . . 16
Outcome . . . 17
Appendix A — PowerShell Scripts 18

Executive Summary — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

When software requests live in email and reclamation never happens, two things break at once: the service desk drowns in untracked asks, and licenses leak — paid for, assigned once, then abandoned on departed users and idle machines. This solution designs a repeatable ServiceNow Software Asset Management (SAM) workflow for the full request lifecycle — intake and triage, entitlement and fulfillment, and automated reclamation of unused licenses — with the catalog items, workflows, and reporting that turn license operations into a measured, auditable process.

This is a complete, deploy-it-today design for running software requests and reclamation as a clean ServiceNow workflow, written for a SAM/ITAM analyst with no prior workflow-design experience. The exact internal configuration of the ServiceNow SAM module (plugins, store apps, console layouts) is deferred to ServiceNow's official documentation; this package gives you the correct, vendor-neutral workflow design and the named artifacts to build it.

What you get

- A plain-language Solution Overview of the request lifecycle (intake to reclamation), catalog items, workflows, entitlement matching, and KPI reporting.
- A Requirements Guide with verify / implement-if-missing for every prerequisite.
- An Implementation Guide of precise, ordered steps to build the catalog items, triage and fulfillment workflows, reclamation rules, and the KPI set — then pilot, validate, and expand.
- Five PowerShell/integration scripts for request and entitlement discovery, fulfillment-data validation, a request and reclamation report, safe remediation of stale entitlements, and a scope/assignment helper (ServiceNow data via the Table API where available).
- Testing and Troubleshooting guides for a confident rollout.

What this delivers

A repeatable request-to-reclamation workflow — a catalog-driven intake, an approval and triage path, an entitlement check before fulfillment, and a scheduled reclamation loop that harvests unused licenses back into the available pool — all measured by a named KPI set so the service desk keeps pace and licenses stop leaking.

The standard deployment model

Assess to Configure to Pilot to Validate to Expand — proven on a pilot product family before you scale.

Outcome

Software requests and reclamation run as a clean, auditable ServiceNow workflow — intake to fulfillment to reclamation — measured, repeatable, and validated on a pilot before fleet-wide rollout.

1. Solution Overview — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

What this solution is

This is a repeatable workflow design for running the full software-license request lifecycle inside ServiceNow Software Asset Management (SAM): a user asks for software, the request is triaged and approved, the system checks whether an entitlement is available, the license is fulfilled and recorded, and — later — unused licenses are automatically reclaimed back into the available pool. It uses ServiceNow's Service Catalog, Flow Designer / workflows, the SAM entitlement and reconciliation engine, discovery/usage data, and reporting. The exact ServiceNow module internals (plugin names, store apps, console menus) are deferred to ServiceNow's official documentation; this solution defines the correct workflow and the named artifacts to build.

Why this matters (the why — covered only here)

Two failure modes cost real money. First, when requests live in email and chat, the service desk has no queue, no SLA, and no audit trail — work is lost, duplicated, and slow. Second, when nobody reclaims licenses, every assignment is permanent: a license handed to a user who leaves, changes role, or simply stops using the product is paid for forever. Over a renewal cycle this is the single largest source of SAM waste. A clean intake-to-reclamation workflow fixes both: it gives requests a measured queue and gives licenses a return path, so the pool you pay for is the pool you actually use.

The request lifecycle

1. Intake — a user submits a request through a published catalog item; the request becomes a tracked record with a requester, product, and justification.
1. Triage — the request is validated, categorized, and routed (auto-approve for low-cost/owned entitlements; manager/owner approval for paid or restricted software).
1. Entitlement check — before fulfillment, the workflow checks the entitlement pool: is a license available, or must one be purchased / reharvested first?
1. Fulfillment — the license is allocated to the user/device, the assignment is recorded against the entitlement, and deployment is triggered or instructed.
1. Reclamation — on a schedule, the workflow finds assignments with no recent usage (or for departed users) and reclaims them, returning the license to the available pool.

Named artifacts (use these exact names everywhere)

The naming convention is `<Purpose> – <Scope> – <Stage>`. Reuse these strings verbatim in every guide and in ServiceNow.

Catalog items:

- `Software Request – Standard – Intake`
- `Software Request – Restricted – Intake`

Workflows / flows:

- `Request Triage – Software – Triage`
- `Entitlement Check – Software – Entitlement`
- `License Fulfillment – Software – Fulfillment`
- `License Reclamation – Unused – Reclamation`

Reclamation rules:

- `Reclaim Rule – Inactive User – Reclamation`
- `Reclaim Rule – No Usage 90d – Reclamation`

Assignment / fulfillment groups:

- `Fulfillment Group – Software – Fulfillment`
- `Approval Group – Restricted Software – Triage`

KPI set:

- `KPI Set – Request & Reclamation – Reporting`

Pilot scope:

- `Pilot Scope – First Product Family – Pilot`

What you can control

- Which products are self-service (standard) versus approval-gated (restricted).
- Triage routing and approval rules per product or cost band.
- The entitlement-match logic: which entitlement record a request consumes.
- Reclamation criteria: inactivity window, departed-user trigger, and grace period.
- The KPI set: request volume, time-to-fulfill, reclamation count, and pool utilization.

Benefits

- Speed — a catalog queue with SLAs replaces untracked email; time-to-fulfill becomes measurable.
- Savings — automated reclamation returns idle licenses, shrinking true-ups and renewals.
- Compliance — every assignment is recorded against an entitlement, so the position is auditable.
- Visibility — one KPI set shows request throughput and reclamation yield over time.

Limitations (be honest with buyers)

- Reclamation accuracy depends on usage/discovery data quality; without it, reclamation is conservative.

- The exact ServiceNow SAM plugin/store-app configuration is product-version specific and deferred to ServiceNow's documentation; this solution gives the workflow design, not the menu clicks.
- SaaS products without a sign-in/usage signal can only be reclaimed on the departed-user trigger.
- Approval routing requires named owners; an unstaffed approval step stalls the queue.

The standard deployment model

> *Build the catalog items and intake, then the triage, entitlement, fulfillment, and reclamation workflows, then the KPI set; pilot on one product family, validate, then expand. Pattern: Assess to Configure to Pilot to Validate to Expand.*

2. Requirements Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

Verify every requirement before configuring. Each entry: Description, Why, Verify, Implement-if-missing. Category-wide prerequisites are in `global-requirements-asset-mgt.md`. ServiceNow-specific module configuration (plugin activation, store apps, console layout) is deferred to ServiceNow's official documentation — verify presence here, follow ServiceNow's docs to enable.

2.1 ServiceNow instance with SAM / ITAM

Description: A ServiceNow instance with the Software Asset Management (and ITAM/CMDB) capability available and licensed. Why: The whole workflow — catalog, entitlements, reconciliation, reclamation — runs on the SAM module. Verify: Sign in to the instance as an admin; confirm the SAM application and its entitlement and reconciliation areas are present. Implement: Activate/license the SAM application per ServiceNow's documentation; do not invent plugin names — follow the vendor's activation steps for your instance version.

2.2 Administrative / SAM roles

Description: An account with the SAM administrator role (or equivalent) plus catalog and flow authoring rights. Why: Building catalog items, workflows, and reclamation rules requires elevated SAM and platform roles. Verify: Confirm the account can open the SAM admin area, the Service Catalog designer, and Flow Designer. Implement: Assign the least-privilege SAM/ITAM admin role plus `catalog_admin` and flow-designer access per ServiceNow's role documentation.

2.3 Entitlement data loaded

Description: Software entitlement records (purchased licenses, quantities, products, and renewal dates) maintained in the SAM module. Why: The entitlement check and fulfillment steps consume a known, authoritative entitlement count; with no entitlements, fulfillment cannot validate availability. Verify: Locate the entitlement quantity and renewal date for a representative product; run `SNOWSAM-Discovery.ps1` to read entitlement records via the Table API. Implement: Import purchase orders / true-up statements into the SAM entitlement records (ServiceNow import sets or the SAM content service), per ServiceNow's documentation.

2.4 Software model / catalog content

Description: Software models (the products you license) defined and mappable to catalog items. Why: A catalog item must reference a real software model so a request maps to an entitlement. Verify: Confirm the products in pilot scope exist as software models; run `SNOWSAM-Validation.ps1`. Implement: Create/normalize the software models for in-scope products (SAM content service or manual), per ServiceNow's documentation.

2.5 Discovery / usage data integration

Description: A source of what is actually installed or used — Discovery/CMDB, an SCCM/Intune integration, or SaaS sign-in/usage telemetry — feeding the SAM reconciliation. Why: Reclamation of unused licenses requires a usage signal; reconciliation needs install data to match entitlements to deployments. Verify: Export an installed-software or last-used report for a sample of devices/users; run `SNOWSAM-Discovery.ps1` to confirm install/usage records are present. Implement: Enable Discovery or the relevant integration and confirm install/usage data lands in the reconciliation tables, per ServiceNow's documentation.

2.6 Approval owners and groups

Description: Named owners for the approval and fulfillment groups — `Approval Group – Restricted Software – Triage` and `Fulfillment Group – Software – Fulfillment`. Why: Restricted requests need an approver, and fulfillment needs an owner; an unstaffed step stalls the queue. Verify: Confirm the groups exist with members and that a test request routes to a real person. Implement: Create the groups, assign members, and set group managers per the named convention.

2.7 Reporting / KPI surface

Description: Access to ServiceNow reporting (or Performance Analytics) plus an external reporting toolchain (Excel / Power BI) for the KPI set. Why: Request throughput, time-to-fulfill, and reclamation yield are delivered as `KPI Set – Request & Reclamation – Reporting`. Verify: You can build a report/dashboard over the request and entitlement tables; the scripts can export CSV for external analysis. Implement: Enable reporting/Performance Analytics access; install Power BI Desktop / current Excel for external dashboards.

2.8 Pilot scope defined

Description: One representative product family chosen as `Pilot Scope – First Product Family – Pilot`. Why: The workflow is validated on a controlled scope before fleet-wide rollout. Verify: A pilot product family and its entitlement/usage data owners are identified. Implement: Choose a high-value, well-understood product family with good usage data as the first pass.

Checklist

- ServiceNow instance with SAM/ITAM available
- SAM admin + catalog + flow authoring roles

- Entitlement data loaded — `SNOWSAM-Discovery.ps1`
- Software models / catalog content for pilot products — `SNOWSAM-Validation.ps1`
- Discovery / usage data integrated for reclamation
- Approval and fulfillment groups staffed
- Reporting / KPI surface available
- `Pilot Scope – First Product Family – Pilot` chosen

3. Implementation Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

Follow these steps in order, in your ServiceNow instance (Service Catalog designer, Flow Designer, and the SAM admin area). ServiceNow module internals (exact menu paths, plugin/store-app names, console layouts) are version-specific and deferred to ServiceNow's official documentation — this guide gives the correct workflow design and the named artifacts; build each artifact using ServiceNow's authoring tools.

> *Model: A catalog-driven intake feeds a triage flow, which calls an entitlement check before a fulfillment flow allocates the license; a scheduled reclamation flow harvests unused licenses back. Pilot on one product family before expanding.*

Step 1 — Create the groups

1. Create the assignment group `Fulfillment Group – Software – Fulfillment` and add the fulfillment team

as members; set a group manager.

1. Create the approval group `Approval Group – Restricted Software – Triage` and add the approvers

(product/license owners); set a group manager.

Step 2 — Build the intake catalog items

1. In the Service Catalog designer, create `Software Request – Standard – Intake` for self-service

(low-cost / already-owned) software. Variables: requester, software model, business justification, target device/user.

1. Create `Software Request – Restricted – Intake` for paid or restricted software, with the same

variables plus a cost-center / approver field.

1. Reference real software models (Requirement 2.4) so each request maps to an entitlement. Publish both

items to the pilot audience only for now.

Step 3 — Build the triage flow

1. In Flow Designer, create `Request Triage – Software – Triage`, triggered when an intake catalog item

is submitted.

1. Logic: validate the request (requester active, software model resolved); categorize standard vs

restricted; for standard with an owned entitlement, auto-approve; for restricted, request approval from `Approval Group – Restricted Software – Triage`.

1. On approval, call `Entitlement Check – Software – Entitlement` (Step 4). On rejection, close the

request with a reason.

Step 4 — Build the entitlement check flow

1. Create `Entitlement Check – Software – Entitlement` as a subflow.

2. Logic: look up the entitlement record for the requested software model; compute available =

purchased minus allocated. If available > 0, return "available" and reserve one unit; if 0, return "needs procurement / reharvest" and route to the SAM owner (and optionally trigger reclamation first).

1. Keep the entitlement-match rule explicit (which entitlement a request consumes when several exist —

e.g. edition/version/cost-center match) so fulfillment is deterministic.

Step 5 — Build the fulfillment flow

1. Create `License Fulfillment – Software – Fulfillment`, called when the entitlement check returns

"available".

1. Logic: allocate the license to the user/device, record the assignment against the entitlement

(decrementing available), assign the task to `Fulfillment Group – Software – Fulfillment`, and trigger or instruct deployment.

1. On completion, mark the request fulfilled and stamp the fulfillment time for the time-to-fulfill KPI.

Step 6 — Build the reclamation rules and flow

1. Create the reclamation rules:

• `Reclaim Rule – Inactive User – Reclamation`: assignment belongs to a user marked inactive /

terminated.

• `Reclaim Rule – No Usage 90d – Reclamation`: assignment has no recorded usage for 90 days (tune the

window per product).

1. Create `License Reclamation – Unused – Reclamation` as a scheduled flow (e.g. daily/weekly). Logic:

evaluate active assignments against both rules; for matches, open a reclamation task, apply a grace period / notification where appropriate, then remove the assignment (returning the unit to available) and record the reclamation event.

1. Start in report-only mode (flag candidates, do not remove) for the first cycle, then enable removal

after the pilot validates the candidates are correct.

Step 7 — Build the KPI set

1. Create `KPI Set – Request & Reclamation – Reporting` with: request volume, average time-to-fulfill,

approval cycle time, reclamation count and reclaimed units, and entitlement pool utilization (available vs allocated) per product.

1. Surface it as a ServiceNow dashboard (or Performance Analytics) and, for external sharing, export via

the scripts to Excel / Power BI.

Step 8 — Apply and validate on the pilot

1. Submit test requests through both catalog items for `Pilot Scope – First Product Family – Pilot`;

confirm triage, entitlement check, and fulfillment behave as designed.

1. Run `SNOWSAM-Validation.ps1` to confirm the catalog items, flows, and entitlement records are present

and consistent via the Table API. Run `SNOWSAM-Report.ps1` for the first KPI snapshot. Complete the Testing Guide.

Step 9 — Enable reclamation, then expand

1. Review the report-only reclamation candidates with the SAM owner; once correct, switch

`License Reclamation – Unused – Reclamation` to enforce (remove + return to pool). Re-run `SNOWSAM-Report.ps1` to confirm reclaimed units appear.

1. Expand catalog publication and the flows to the next product families only after the pilot passes.

Use `SNOWSAM-Set-ScopeTargeting.ps1` to record/expand which products and groups are in scope.

Order of operations

```
Groups -> Catalog items -> Triage flow -> Entitlement check -> Fulfillment flow -> Reclamation rules & flow -> KPI set -> Pilot validate -> Enable reclamation -> Expand
```

5. Testing Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

Run after the pilot build applies (Implementation Step 8) and after each workflow change. Use the scripts where noted; they read ServiceNow via the Table API where available.

Test 1 — Intake captures a tracked request

Do: Submit `Software Request – Standard – Intake` and `Software Request – Restricted – Intake` for a pilot product. Expected: Each submission creates a tracked request record with requester, software model, and justification — no email needed.

Test 2 — Triage routes correctly

Do: Observe `Request Triage – Software – Triage` for both submissions. Expected: The standard request with an owned entitlement auto-approves; the restricted request routes to `Approval Group – Restricted Software – Triage` and waits for approval.

Test 3 — Entitlement check is accurate

Do: Run `SNOWSAM-Validation.ps1` and inspect the entitlement the request will consume. Expected: `Entitlement Check – Software – Entitlement` returns "available" when units exist and "needs procurement / reharvest" when the pool is exhausted; available = purchased minus allocated matches the record.

Test 4 — Fulfillment records the assignment

Do: Approve and let `License Fulfillment – Software – Fulfillment` complete. Expected: The license is allocated, the assignment is recorded against the entitlement (available decremented), the request is marked fulfilled, and a fulfillment time is stamped.

Test 5 — Reclamation flags the right candidates (report-only)

Do: Run `License Reclamation – Unused – Reclamation` in report-only mode; review with the SAM owner; run `SNOWSAM-Report.ps1`. Expected: `Reclaim Rule – Inactive User – Reclamation` and `Reclaim Rule – No Usage 90d – Reclamation` flag only genuinely inactive/unused assignments — no false positives on active users.

Test 6 — Reclamation returns units (enforce mode)

Do: Switch reclamation to enforce on a known-stale assignment; re-run `SNOWSAM-Report.ps1`. Expected: The assignment is removed, the unit returns to the available pool, and a reclamation event is recorded.

Test 7 — KPI set reports throughput and yield

Do: Run `SNOWSAM-Report.ps1` (and open the dashboard for `KPI Set – Request & Reclamation – Reporting`). Expected: Request volume, time-to-fulfill, reclamation count/units, and pool utilization render with real pilot data.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1 to 5 pass; a full request flows intake to fulfillment and reclamation candidates are correct in report-only
Expand pass	Tests 6 to 7 pass; reclamation returns units and the KPI set reports cleanly before expanding scope

6. Troubleshooting Guide — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

Use `SNOWSAM-Validation.ps1` / `SNOWSAM-Discovery.ps1` for state, `SNOWSAM-Report.ps1` for KPIs, and `SNOWSAM-Remediation.ps1` (default `-WhatIf`) for safe fixes. ServiceNow module internals are deferred to ServiceNow's documentation; the table below addresses the workflow design.

A. Requests not flowing through triage

Symptom	Cause	Remediation
Request submitted but never triaged	`Request Triage – Software – Triage` not active or trigger mismatched	Confirm the flow is active and its trigger matches the catalog item; run `SNOWSAM-Validation.ps1`
Restricted request never approved	`Approval Group – Restricted Software – Triage` empty or unstaffed	Add members and a group manager; re-test routing
Standard request asks for approval	Auto-approve rule not matching the owned entitlement	Check the triage categorization and the entitlement-match rule

B. Entitlement check wrong

Symptom	Cause	Remediation
"Available" when pool is exhausted	available not computed as purchased minus allocated	Fix `Entitlement Check – Software – Entitlement` math; reconcile entitlement records
Consumes the wrong entitlement	Entitlement-match rule ambiguous when several records exist	Make the match rule deterministic (edition / version / cost-center)

Symptom	Cause	Remediation
No entitlement found	Software model not linked to an entitlement	Link the model to the entitlement record; run `SNOWSAM-Discovery.ps1`

C. Fulfillment not recording

- Confirm `License Fulfillment – Software – Fulfillment` decrements available and stamps fulfillment time.
- If the task never assigns, check `Fulfillment Group – Software – Fulfillment` membership.

D. Reclamation flags active users (false positives)

Symptom	Cause	Remediation
Active user flagged for reclaim	No/stale usage signal feeding `Reclaim Rule – No Usage 90d – Reclamation`	Confirm discovery/usage data is current; lengthen the window; keep report-only until clean
Departed user not flagged	`Reclaim Rule – Inactive User – Reclamation` not reading the HR/identity status	Confirm the inactive/terminated source field; map it into the rule
Reclaim removed a needed license	No grace period / notification	Add a grace period and assignee notification before removal

E. KPI set empty or wrong

- `SNOWSAM-Report.ps1` returns nothing → confirm Table API access and that requests/entitlements exist in

scope; check the instance URL and credentials.

- Numbers look off → confirm time-to-fulfill uses the fulfillment timestamp from Step 5 and reclamation

events are recorded.

F. Diagnostics quick reference

- Run `SNOWSAM-Discovery.ps1` to dump request, entitlement, and usage records via the Table API.
- Run `SNOWSAM-Validation.ps1` to confirm catalog items, flows, and entitlements are present.
- Run `SNOWSAM-Remediation.ps1` with the default `-WhatIf` to preview stale-entitlement fixes before

applying.

G. When to escalate

If the flows are active and staffed, entitlement and usage data are current, the scripts confirm records are present, and requests still stall or reclamation still misfires, capture the script output and the flow execution details and open a ServiceNow support case (module-internal behavior is vendor-supported).

7. Best Practices — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

- Make every request a catalog item. Replace email and chat with `Software Request – Standard – Intake`

and `Software Request – Restricted – Intake` so every ask is tracked, routed, and measured — the single most important practice.

- Check entitlement before you fulfill. Always run `Entitlement Check – Software – Entitlement` before

allocation so you never over-allocate beyond the purchased pool.

- Reclaim on a schedule, not by hope. `License Reclamation – Unused – Reclamation` running on a cadence

is what stops license leakage; manual reclamation never happens.

- Start reclamation in report-only. Validate `Reclaim Rule – Inactive User – Reclamation` and

`Reclaim Rule – No Usage 90d – Reclamation` candidates with the SAM owner before enforcing removal — earns trust and avoids reclaiming an active user's license.

- Tune the inactivity window per product. 90 days is a sane default; long-cycle or seasonal tools need a

longer window, high-cost SaaS a shorter one.

- Keep the entitlement-match rule deterministic. Define exactly which entitlement a request consumes when

several exist (edition / version / cost-center) so fulfillment and reporting are repeatable.

- Staff every approval and fulfillment group. An unstaffed `Approval Group – Restricted Software – Triage`

or `Fulfillment Group – Software – Fulfillment` stalls the whole queue.

- Add a grace period and notification before removal. Warn the assignee before reclaiming so a still-needed

license can be retained — reduces re-request churn.

- Measure with one KPI set. Track `KPI Set – Request & Reclamation – Reporting` (time-to-fulfill,

reclaimed units, pool utilization) every cycle and chase the laggards.

- Pilot, then expand. Prove the workflow on `Pilot Scope – First Product Family – Pilot` before publishing

catalog items fleet-wide.

• Document the cadence. Record reclamation windows, approval rules, and the entitlement-match logic for

audits and repeatability.

8. Deliverables Summary — ServiceNow SAM Request Fulfillment & License Reclamation Workflow

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The request lifecycle (intake to reclamation), catalog items, workflows, entitlement matching, KPIs, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Asset Mgt)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: groups, catalog items, triage/entitlement/fulfillment/reclamation flows, KPI set
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

PowerShell / integration scripts

Script	What it does
`SNOWSAM-Discovery.ps1`	Reads request, entitlement, and usage records via the ServiceNow Table API (read-only)
`SNOWSAM-Validation.ps1`	Confirms catalog items, flows, software models, and entitlement records are present and consistent (read-only)
`SNOWSAM-Report.ps1`	Builds the request and reclamation KPI report; CSV export (read-only)
`SNOWSAM-Remediation.ps1`	Safe remediation of stale entitlements/assignments; default `-WhatIf` (preview only)
`SNOWSAM-Set-ScopeTargeting.ps1`	Records/expands which products and groups are in scope for the workflow

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell / integration scripts in a single ZIP.
- A repeatable process following the model: Assess to Configure to Pilot to Validate to Expand.

Outcome

Software requests and reclamation run as a clean, auditable ServiceNow workflow — intake to fulfillment to reclamation — measured by a single KPI set and validated on a pilot before fleet-wide rollout.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SNOWSAM-Discovery.ps1

Purpose: Reads the current SAM posture from a ServiceNow instance via the REST Table API: open software requests, software entitlement records (purchased vs allocated), and software usage/install records. Read-only — it makes only GET calls and changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Discovery.ps1 ` -InstanceUrl https://.service-now.com -Credential (Get-Credential) Optionally pass -EntitlementTable / -RequestTable / -UsageTable to match your instance's tables. EXPECTED RESULTS A summary count of open requests, entitlement records with available = purchased - allocated, and usage records, plus an optional CSV export. .NOTES Read-only. ServiceNow table/field names are instance-specific — the defaults below are generic placeholders. Confirm the correct table names for your instance in ServiceNow's Table API docs and override with the parameters; do not assume these tables exist as-is. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)] [pscredential]$Credential, # INSTANCE-SPECIFIC: replace with the actual table names in your ServiceNow SAM instance. [string]$RequestTable = 'sc_req_item', [string]$EntitlementTable = 'samp_sw_entitlement', [string]$UsageTable = 'samp_sw_install', [int]$Limit = 200, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Discovery.ps1 ` -InstanceUrl https://<your-instance>.service-now.com -Credential (Get-Credential) Optionally pass -EntitlementTable / -RequestTable / -UsageTable to match your instance's tables. EXPECTED RESULTS A summary count of open requests, entitlement records with available = purchased - allocated, and usage records, plus an optional CSV export. .NOTES Read-only. ServiceNow table/field names are instance-specific — the defaults below are generic placeholders. Confirm the correct table names for your instance in ServiceNow's Table API docs and override with the parameters; do not assume these tables exist as-is. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)] [pscredential]$Credential, # INSTANCE-SPECIFIC: replace with the actual table names in your ServiceNow SAM instance. [string]$RequestTable = 'sc_req_item', [string]$EntitlementTable = 'samp_sw_entitlement', [string]$UsageTable = 'samp_sw_install', [int]$Limit = 200, [string]$CsvPath)

SNOWSAM-Remediation.ps1

Purpose: Identifies stale software assignments (no recorded usage past a threshold, or belonging to an inactive user) and previews releasing them back to the available pool. By DEFAULT it runs in -WhatIf mode and changes NOTHING — it only lists what it would reclaim. You must explicitly pass -WhatIf:$false to perform updates via the ServiceNow REST Table API. HOW TO RUN Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Remediation.ps1 ` -InstanceUrl https://.service-now.com -Credential (Get-Credential) Apply (after review): ... -WhatIf:$false EXPECTED RESULTS In preview: a list of candidate stale assignments and the action that would be taken. In apply mode: each candidate is released and the action is confirmed. .NOTES Defaults to -WhatIf (preview). Assignment table/field names are instance-specific placeholders — confirm in ServiceNow's Table API docs and override via parameters. Run SNOWSAM-Report.ps1 first to review reclamation candidates; prefer the in-platform "License Reclamation - Unused - Reclamation" flow for production reclamation. This script is a manual safety-net for one-off cleanup. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)]

[pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override these table/field names for your instance. [string]$AssignmentTable = 'samp_sw_allocation', [int]$InactiveDays = 90, [switch]$WhatIf = $true # default ON: preview only)

How to run: Preview (safe, default): powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Remediation.ps1 ` -InstanceUrl https://<your-instance>.service-now.com -Credential (Get-Credential) Apply (after review): ... -WhatIf:$false EXPECTED RESULTS In preview: a list of candidate stale assignments and the action that would be taken. In apply mode: each candidate is released and the action is confirmed. .NOTES Defaults to -WhatIf (preview). Assignment table/field names are instance-specific placeholders — confirm in ServiceNow's Table API docs and override via parameters. Run SNOWSAM-Report.ps1 first to review reclamation candidates; prefer the in-platform "License Reclamation - Unused - Reclamation" flow for production reclamation. This script is a manual safety-net for one-off cleanup. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)] [pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override these table/field names for your instance. [string]$AssignmentTable = 'samp_sw_allocation', [int]$InactiveDays = 90, [switch]$WhatIf = $true # default ON: preview only)

SNOWSAM-Report.ps1

Purpose: Builds the "KPI Set - Request & Reclamation - Reporting" snapshot from ServiceNow via the REST Table API: request volume and state breakdown, average time-to-fulfill, reclamation count and reclaimed units, and entitlement pool utilization (available vs allocated). Read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Report.ps1 ` -InstanceUrl https://.service-now.com -Credential (Get-Credential) ` -CsvPath C:\Temp\snowsam-kpi.csv EXPECTED RESULTS Console KPI summary plus an optional per-entitlement CSV (product, purchased, allocated, available, utilization%). .NOTES Read-only. Table/field names are instance-specific placeholders — confirm in ServiceNow's Table API docs and override via parameters. Time-to-fulfill assumes a fulfillment timestamp is stamped at fulfillment (see Implementation Step 5). #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)] [pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override these table names for your instance. [string]$RequestTable = 'sc_req_item', [string]$EntitlementTable = 'samp_sw_entitlement', [string]$ReclamationTable = 'samp_sw_reclamation', [int]$Limit = 1000, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Report.ps1 `-InstanceUrl https://<your-instance>.service-now.com -Credential (Get-Credential)` -CsvPath C:\Temp\snowsam-kpi.csv EXPECTED RESULTS Console KPI summary plus an optional per-entitlement CSV (product, purchased, allocated, available, utilization%). .NOTES Read-only. Table/field names are instance-specific placeholders — confirm in ServiceNow's Table API docs and override via parameters. Time-to-fulfill assumes a fulfillment timestamp is stamped at fulfillment (see Implementation Step 5). #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)] [pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override these table names for your instance. [string]$RequestTable = 'sc_req_item', [string]$EntitlementTable = 'samp_sw_entitlement', [string]$ReclamationTable = 'samp_sw_reclamation', [int]$Limit = 1000, [string]$CsvPath)

SNOWSAM-Set-ScopeTargeting.ps1

Purpose: Maintains the targeting scope for the request-to-reclamation workflow: which software products (models) and which catalog audiences / groups the workflow currently covers. Use it to define the pilot

scope ("Pilot Scope - First Product Family - Pilot") and to expand to further product families after the pilot passes. Writes a local scope manifest (JSON) and, optionally, verifies the listed products and groups exist in ServiceNow via the read-only REST Table API. HOW TO RUN Define/expand the local scope manifest: powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Set-ScopeTargeting.ps1 ` -Products 'Product Family A','Product Family B' ` -Groups 'Fulfillment Group - Software - Fulfillment','Approval Group - Restricted Software - Triage' ` -ScopePath .\snowsam-scope.json Optionally verify against the instance (read-only): ... -InstanceUrl https://.service-now.com -Credential (Get-Credential) -Verify EXPECTED RESULTS A written/updated scope manifest listing in-scope products and groups; with -Verify, a FOUND/MISSING check of each against ServiceNow (read-only — no changes to the instance). .NOTES The manifest write is local and safe. -Verify makes read-only GET calls only. Table/field names are instance-specific placeholders — confirm in ServiceNow's Table API docs and override via parameters. Actual catalog audience/scope changes are made in the Service Catalog designer (see Implementation Guide Step 9); this script records intent and verifies prerequisites, it does not re-scope catalog items in the platform. #> [CmdletBinding()] param([string[]]$Products = @(), [string[]]$Groups = @(), [string]$ScopePath = '.\snowsam-scope.json', [switch]$Verify, [string]$InstanceUrl, [pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override for -Verify. [string]$SoftwareModelTable = 'cmdb_software_product_model', [string]$GroupTable = 'sys_user_group')

How to run: Define/expand the local scope manifest: powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Set-ScopeTargeting.ps1 `-Products 'Product Family A','Product Family B'` -Groups 'Fulfillment Group - Software - Fulfillment','Approval Group - Restricted Software - Triage' ` -ScopePath .\snowsam-scope.json Optionally verify against the instance (read-only): ... -InstanceUrl https://<your-instance>.service-now.com -Credential (Get-Credential) -Verify EXPECTED RESULTS A written/updated scope manifest listing in-scope products and groups; with -Verify, a FOUND/MISSING check of each against ServiceNow (read-only — no changes to the instance). .NOTES The manifest write is local and safe. -Verify makes read-only GET calls only. Table/field names are instance-specific placeholders — confirm in ServiceNow's Table API docs and override via parameters. Actual catalog audience/scope changes are made in the Service Catalog designer (see Implementation Guide Step 9); this script records intent and verifies prerequisites, it does not re-scope catalog items in the platform. #> [CmdletBinding()] param([string[]]$Products = @(), [string[]]$Groups = @(), [string]$ScopePath = '.\snowsam-scope.json', [switch]$Verify, [string]$InstanceUrl, [pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override for -Verify. [string]$SoftwareModelTable = 'cmdb_software_product_model', [string]$GroupTable = 'sys_user_group')

SNOWSAM-Validation.ps1

Purpose: Verifies the named workflow artifacts exist and are consistent in ServiceNow via the REST Table API: the intake catalog items, the triage/entitlement/fulfillment/reclamation flows, the software models, and the entitlement records. Read-only — only GET calls. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Validation.ps1 ` -InstanceUrl https://.service-now.com -Credential (Get-Credential) EXPECTED RESULTS "WORKFLOW PRESENT" when the expected catalog items, flows, and entitlement records are found; otherwise a list of what is missing. .NOTES Read-only. Catalog/flow/entitlement table names are instance-specific — the defaults are generic placeholders. Confirm them in ServiceNow's Table API docs and override via parameters. Artifact names follow the convention - - used in the implementation guide. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)] [pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override these table names for your instance. [string]$CatalogItemTable = 'sc_cat_item', [string]$FlowTable = 'sys_hub_flow', [string]$EntitlementTable = 'samp_sw_entitlement')

How to run: powershell -ExecutionPolicy Bypass -File .\SNOWSAM-Validation.ps1 ` -InstanceUrl https://<your-instance>.service-now.com -Credential (Get-Credential) EXPECTED RESULTS "WORKFLOW PRESENT" when the expected catalog items, flows, and entitlement records are found; otherwise a list of what is missing. .NOTES Read-only. Catalog/flow/entitlement table names are instance-specific — the defaults are generic placeholders. Confirm them in ServiceNow's Table API docs and override via parameters. Artifact names follow the convention <Purpose> - <Scope> - <Stage> used in the implementation guide. #> [CmdletBinding()] param([Parameter(Mandatory)] [string]$InstanceUrl, [Parameter(Mandatory)] [pscredential]$Credential, # INSTANCE-SPECIFIC: confirm/override these table names for your instance. [string]$CatalogItemTable = 'sc_cat_item', [string]$FlowTable = 'sys_hub_flow', [string]$EntitlementTable = 'samp_sw_entitlement')